

# 6.6 - Prometheus e a linguagem PromQL

Coleta por pull, séries temporais e consultas

# Um label derrubou o monitoramento

- **Dev adicionou o label `user_id` nas métricas HTTP**
  - A plataforma tinha 80 mil usuários ativos

## LUNALOG

No primeiro mês com Prometheus, um desenvolvedor da LunaLog teve uma ideia que parecia ótima: adicionar um label `user_id` às métricas de requisição HTTP, para analisar por usuário. Com 80 mil usuários ativos, o Prometheus passou a criar uma série temporal por usuário, multiplicada por endpoint e por código de status. Em três dias, o banco de métricas explodiu e começou a degradar o próprio sistema de monitoramento. A ferramenta que deveria vigiar a saúde do sistema virou a paciente mais doente. Cardinalidade de labels é o erro número um de quem começa com Prometheus.

# Como o Prometheus coleta e armazena

Modelo pull e séries temporais com labels

- **Prometheus busca métricas por pull, não recebe push**
  - Ele raspa um endpoint HTTP de cada alvo periodicamente
- **Cada métrica é uma série identificada por labels**
  - Nome da métrica mais o conjunto de labels define a série
- **O alvo expõe métricas em formato de texto simples**
  - Bibliotecas client cuidam desse formato para você
- **Alvos são descobertos por config ou service discovery**
  - em Kubernetes a descoberta é dinâmica via anotações

# PromQL para as métricas RED

PROMQL

```
# Rate: requisicoes por segundo nos ultimos 5 min
sum(rate(http_requests_total[5m])) by (service)

# Errors: proporcao de respostas 5xx
sum(rate(http_requests_total{status=~'5..'}[5m]))
  / sum(rate(http_requests_total[5m]))

# Duration: latencia no percentil 95
histogram_quantile(0.95,
  sum(rate(http_request_duration_seconds_bucket[5m])) by (le))
```

# Cardinalidade tem custo concreto

- Cada valor único de label cria uma série nova
- Labels devem ter domínio pequeno e finito

Nunca use identificadores de alta cardinalidade como label: `user_id`, `request_id`, e-mail ou IP. Eles multiplicam séries sem limite e podem derrubar o Prometheus, exatamente como aconteceu com a LunaLog. Esses valores pertencem aos logs e aos traces.

- Para detalhe por requisição, recorra ao trace, não à métrica



O Prometheus coleta e o PromQL responde. Mas um número numa consulta não comunica nada para quem está de plantão às duas da manhã. Dado sem visualização é dado invisível. A próxima ferramenta transforma séries temporais em painéis, e aqui mora um paradoxo cruel: um dashboard com 47 painéis durante um incidente pode ser tão inútil quanto não ter dashboard nenhum.

**Grafana: design de dashboards que comunicam**